Projet d'expertise en Statistique et Probabilités :

Chaînes de Markov : Quand un jeu de cartes est-il bien mélangé ?

2022-2023



Auteur : SOULEYMAN Mahamat Youssouf

Professeur encadrant : Luis Fredes

Table des matières

1 Introduction **3**

2 Présentation du problème **4**

3 Top to random shuffle **5**

3.1 Illustration du mélange dans le cas de S_3 . . . 5
3.2 Modélisation mathématique d'une étape du top-to-random shuffle. . . . 5
3.3 Modélisation comme chaîne de Markov . . . 5
3.4 Propriétés de la chaîne de Markov . . . 6
3.4.1 Irréductibilité et Apériodicité . . . 6
3.4.2 Distribution stationnaire . . . 7
3.5 Distance en variation totale . . . 8
3.6 Temps de mélange . . . 8
3.7 Borne supérieure pour le temps de mélange . . . 8
3.7.1 Problème du collecteur de coupons . . . 9
3.7.2 Filtration et temps d'arrêt . . . 10
3.7.3 Contrôle de la convergence par TFS . . . 10
3.8 Borne inférieure pour le *Top-to-Random Shuffle* . . . 12
3.9 Simulations pour observer la convergence . . . 13
3.9.1 Le phénomène de cutoff pour le mélange top-to-random . . . 13
3.9.2 Convergence vers la distribution uniforme . . . 14
3.9.3 Distance en variation totale et temps de mélange . . . 15
3.10 Analyse globale et critique de la méthode top-to-random shuffle . . . 17

4 Perspectives **17**

5 Conclusion **18**

Annexes **21**

A Démonstration de la proposition sur la distance en variation **21**

B Annexe : Code source des simulations **22**

1 Introduction

Lorsqu'on prend un nouveau jeu de cartes, celui-ci est parfaitement ordonné, ce qui amène naturellement à s'interroger sur la manière et le nombre de mélanges nécessaires pour le rendre suffisamment aléatoire. Ce problème, à la croisée des mathématiques et de la théorie des probabilités, trouve une réponse dans l'étude des chaînes de Markov.

Pour répondre à cette question, nous modéliserons l'arrangement des cartes comme les états d'une chaîne de Markov, où la matrice de transition est définie par la règle de mélange employée. L'analyse de cette chaîne permet d'étudier la convergence vers la mesure stationnaire, qui correspond à un état d'équilibre distributionnel où toutes les permutations du jeu sont équiprobables.

Cette notion de mélange est utilisée dans des domaines aussi variés que la cryptographie, les jeux de hasard et la simulation informatique.

Ce rapport a pour objectif d'examiner une technique spécifique de mélange de cartes, appelée Top-to-Random Shuffle, et d'analyser le nombre d'itérations nécessaires pour atteindre un état proche de l'uniformité. Des simulations numériques seront réalisées pour illustrer ce modèle.

2 Présentation du problème

Si on prend un jeu de cartes avec N cartes, chacune ayant une position unique dans l'ordre initial. Un mélange a pour but de randomiser cet ordre pour s'assurer qu'aucune carte ne se trouve dans une position prévisible après un nombre suffisant de mélanges. La question centrale est donc de déterminer à partir de combien de mélanges d'une méthode donnée , on peut considérer que les cartes sont "bien mélangées". Autrement dit, on cherche à savoir à partir de combien d'itérations de mélange la distribution des cartes devient suffisamment proche de la distribution uniforme, dans laquelle chaque permutation du jeu a la même probabilité d'apparaître.

D'un point de vue formel, nous modélisons l'ensemble des arrangements possibles du jeu de cartes par le groupe symétrique S_N , qui est l'ensemble des $N!$ permutations possibles des N cartes. Une méthode de mélange est alors un processus stochastique qui, à chaque étape, applique une transformation probabiliste sur une permutation donnée. Cela définit une chaîne de Markov sur S_N , où la matrice de transition M de cette chaîne décrit les probabilités de passage d'une permutation σ à σ'.

L'objectif est donc d'évaluer le nombre minimal d'itérations de la chaîne associée au mécanisme de mélange pour que la distribution de la permutation resultante de la chaîne soit suffisamment proche de la distribution uniforme. Cette proximité peut être mesurée à l'aide d'une métrique appelée **la distance de variation totale** (voir def...3.5) entre la distribution actuelle et la distribution uniforme. Plus cette distance est petite, plus le jeu est proche d'un état parfaitement mélangé. Le nombre d'itérations nécessaire pour atteindre ce seuil de proximité est appelé **le temps de mélange**. Ce temps dépend de la méthode de mélange utilisée et de la taille du jeu (le nombre de cartes).

Dans les sections suivantes, nous modéliserons la méthode de mélange appelé Top to random shuffle. Pour ce modèle nous étudierons ces propriétés de convergences , son temps de mélange et vérifierons ensuite par des simulations.

3 Top to random shuffle

On note S_N l'ensemble des bijections de $[1, N]$ dans $[1, N]$.C'est le groupe symétrique d'ordre N pour la composition des bijections. Notons que $|S_N| = N!$.

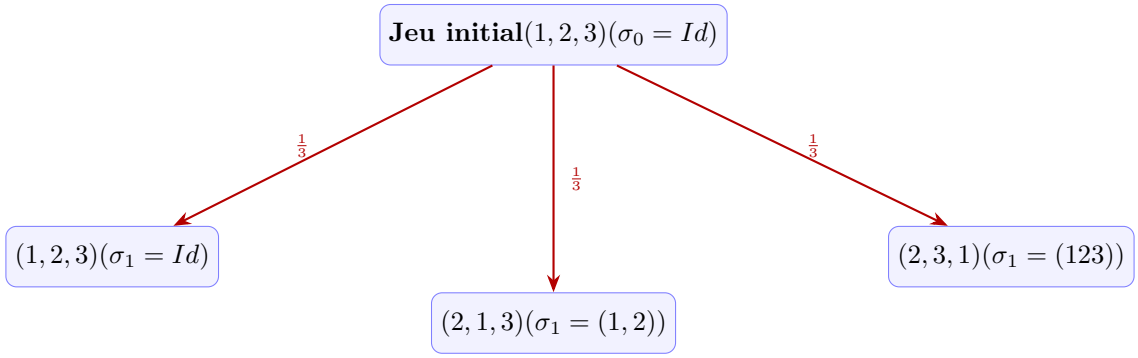
Considérons un jeu de N cartes numérotées de 1 à N, initialement ordonnées (1 étant la carte du dessus). On identifie un ordre de carte avec un élément de S_N.

Le Top to random shuffle consiste à :

1. Prendre la carte du dessus
2. La réinsérer uniformément au hasard dans le jeu (y compris éventuellement à sa position initiale)

3.1 Illustration du mélange dans le cas de S_3

Voici une illustration simple du mécanisme dans le cas de S_3 ($N = 3$). La configuration initiale du jeu est l'identité.



Explication du schéma :

- ▷ La carte du dessus (1) est déplacée vers l'une des 3 positions
- ▷ Les probabilités de transition sont uniformes ($\frac{1}{3}$)
- ▷ L'état $(2, 3, 1)$ correspond à une insertion en position 3

FIGURE 1 – Top to Random Shuffle pour $n = 3$: transitions possibles en une étape depuis l'état initial.

3.2 Modélisation mathématique d'une étape du top-to-random shuffle.

Chaque mélange correspond à une permutation des cartes.

Chaque opération de mélange transforme la permutation courante σ en une nouvelle permutation σ' de la manière suivante :

— On note C_k la permutation cyclique $(k, k-1, \ldots, 2, 1)$
— Si on insère la carte du dessus à la position k, alors $\sigma' = C_k \circ \sigma$

La position d'insertion k est choisie uniformément dans $\{1, \ldots, N\}$, indépendamment des mélanges précédents.

3.3 Modélisation comme chaîne de Markov

Proposition 1. *Soit $\sigma_0 = (1, 2, \ldots, N)$ la permutation initiale. Soit $(X_n)_{n\geq 0}$ la suite des permutations successives du jeu. Alors (X_n) est une chaîne de Markov homogène à temps discret sur S_N.*

Démonstration. — L'espace d'états S_N est fini

— La propriété de Markov est satisfaite :

$$\mathbb{P}(X_n = \sigma_n \mid X_{n-1} = \sigma_{n-1}, \ldots, X_0 = \sigma_0) = \mathbb{P}(X_n = \sigma_n \mid X_{n-1} = \sigma_{n-1})$$

— Pour chaque transition, il existe exactement N permutations possibles σ_n accessibles depuis σ_{n-1}, chacune avec probabilité $\frac{1}{N}$:

$$M_{\sigma,\sigma'} = \begin{cases} \frac{1}{N} & \text{si } \exists k \in \{1, \ldots, N\} \text{ tel que } \sigma' = C_k \circ \sigma \\ 0 & \text{sinon} \end{cases}$$

□

3.4 Propriétés de la chaîne de Markov

Dans cette section, nous étudierons les propriétés de la chaîne de Markov qui nous permettrons en suite de montrer la convergence de la chaîne.

3.4.1 Irréductibilité et Apériodicité

Théorème 1. *La chaîne de Markov $(X_n)_{n\geq 0}$ est irréductible et apériodique.*

Démonstration. La chaîne est irréductible car toute permutation peut être atteinte depuis toute autre permutation. Pour le démontrer :

1. On sait que le groupe symétrique S_N est engendré par :
   — La transposition $\tau = (2, 1)$ (qui échange les deux premières cartes)
   — Le cycle complet $C_N = (N, N-1, \ldots, 2, 1)$
   Ainsi, $S_N = \langle \tau, C_N \rangle$.
2. Soient σ et σ' deux permutations quelconques dans S_N. Il existe une décomposition :

   $$\sigma^{-1} \circ \sigma' = \alpha_1 \circ \alpha_2 \circ \cdots \circ \alpha_r$$

   où chaque α_i est soit τ, soit C_N, soit C_N^{-1}.
3. Dans notre chaîne de Markov :
   — L'action C_k correspond à insérer la carte du dessus à la position k
   — La probabilité de transition entre deux états est donnée par :

   $$\begin{aligned} \mathbb{P}(X_r = \sigma' \mid X_0 = \sigma) = \mathbb{P}(&X_r = \sigma' \mid X_{r-1} = \alpha_1\sigma) \\ &\times \mathbb{P}(X_{r-1} = \alpha_1\sigma \mid X_{r-2} = \alpha_2\alpha_1\sigma) \\ &\times \cdots \\ &\times \mathbb{P}(X_1 = \alpha_{r-1} \cdots \alpha_1\sigma \mid X_0 = \sigma) \end{aligned}$$

   Par la propriété de Markov, cette probabilité se factorise en produit des probabilités de transition élémentaires.
   — Chaque étape élémentaire correspondant à un générateur a une probabilité :

   $$\mathbb{P}(X_k = \alpha_k\sigma_k \mid X_{k-1} = \sigma_k) \geq \frac{1}{N}$$

   car :
   — Pour $\alpha_k = C_N$: probabilité exactement $\frac{1}{N}$
   — Pour $\alpha_k = \tau$: nécessite au plus 2 étapes (probabilité $\geq \frac{1}{N^2}$)
4. Par conséquent, pour toute paire (σ, σ'), il existe $r \leq 2N(N-1)$ tel que :

   $$\mathbb{P}(X_r = \sigma' \mid X_0 = \sigma) \geq \left(\frac{1}{N}\right)^r > 0$$

La chaîne est apériodique car :

$$\mathbb{P}(X_n = \sigma \mid X_{n-1} = \sigma) = \frac{1}{N} > 0 \quad \text{(en choisissant de remettre la carte du dessus à sa position initiale)}$$

Ceci implique que le plus grand commun diviseur des longueurs des cycles est 1. □

Selon (1) toute chaîne de Markov irréductible à espace d'état fini est récurrente positive. Par conséquent il existe une seule distribution stationnaire. Ça sera l'objet de la section suivante.

3.4.2 Distribution stationnaire

Définition 1 (Loi uniforme)**.** *La loi uniforme sur S_N est une application de $\mathcal{P}(S_N)$ vers $[0,1]$, définie par :*

$$\begin{cases} \mathcal{P}(S_N) & \longrightarrow [0,1] \\ A & \longmapsto \dfrac{\text{Card}(A)}{N!} \end{cases}$$

Il s'agit d'une loi de probabilité sur $(S_N, \mathcal{P}(S_N))$.

Proposition 2. *La mesure invariante π de la chaîne du top-to-random shuffle (avec matrice de transition M) est la mesure uniforme sur S_N et vers laquelle elle converge en loi.*

Démonstration. — Montrons que la mesure uniforme est l'unique mesure invariante de la chaîne .
Soit $\sigma \in S_N$

$$(\pi M)(\sigma) = \sum_{\alpha \in S_N} \pi(\sigma) M(\alpha, \sigma) = \sum_{\alpha \in S_N} \frac{1}{N!} M(\alpha, \sigma)$$

En posant $\alpha = k^{-1}\sigma$ alors $k = \sigma\alpha^{-1}$, par suite :

$$(\pi M)(\sigma) = \frac{1}{N!} \sum_{k \in S_N} M(k^{-1}\sigma, \sigma) = \frac{1}{N!} \sum_{k \in S_N} \frac{1}{N} = \frac{1}{N!} \times 1$$

en effet $M(k^{-1}\sigma, \sigma)$ est une transition de la chaîne et k parcours S_N uniformément : il y a exactement N possibilité d'insérer la carte dans une position.

— Montrons que la chaîne converge en loi vers π.

Dans la proposition 2. M est irréductible , récurrente positive et apériodique , alors par le théorème de convergence en loi , la chaîne converge vers la mesure de probabilité invariante qui est la mesure uniforme π.

$$\lim_{t \to \infty} \mathbb{P}(X_t = \sigma) = \pi(\sigma), \quad \text{pour tout } \sigma \in S_N,$$

où π est la mesure uniforme sur S_n.

□

Formalisme d'un jeu "bien mélangé"

Maintenant, l'enjeu de l'étude est de déterminer combien de fois il faut mélanger pour que le paquet de cartes soit considéré comme bien mélangé. L'objet de cette partie est donc d'étudier la vitesse de convergence de la méthode de mélange le top-to-random shuffle.

Pour cela, nous allons utiliser la convergence en loi présentée dans la proposition 2 afin d'établir formellement cette notion de mélange.

Plus précisément, nous allons introduire une distance entre distributions de probabilité pour quantifier à quel point la distribution des configurations du jeu après un certain nombre de mélanges est proche de la loi uniforme. Ensuite, nous allons définir mathématiquement la notion d'un jeu bien mélangé.

Pour quantifier la proximité entre deux distributions de probabilité sur les permutations, nous introduisons la distance en variation totale.

3.5 Distance en variation totale

Soient μ et ν deux mesures de probabilité sur S_N. La distance en variation entre μ et ν est le réel $d_V(\mu, \nu) \in [0, 1]$ défini par :

$$d_V(\mu, \nu) = \max_{A \subset \mathcal{P}(S_N)} |\mu(A) - \nu(A)|. \tag{1}$$

Cette définition est assez intuitive : elle mesure la différence maximale entre les probabilités que μ et ν attribuent aux mêmes ensembles d'états. Il s'agit bien d'une distance sur l'ensemble des mesures de probabilité définies sur S_N.

L'intérêt principal de cette distance est son bon comportement lors du passage à la limite. En effet, si l'on note, pour tout $n \in \mathbb{N}$, μ_n la loi de probabilité de X_n, on a :

$$\lim_{n \to +\infty} d_V(\mu_n, \pi) = 0, \tag{2}$$

ce qui exprime la convergence en loi vers la distribution uniforme π.

Cela découle immédiatement de la proposition suivante.

Proposition 3. *Pour toutes mesures de probabilité μ et ν sur S_N, on a :*

$$d_V(\mu, \nu) = \frac{1}{2} \sum_{\sigma \in S_N} |\mu(\{\sigma\}) - \nu(\{\sigma\})|. \tag{3}$$

Cette formulation explicite, en somme, permet un calcul pratique de la distance en variation et servira de base à l'étude du temps de mélange.

La démonstration complète de cette proposition est reportée en **A**.

Munis de cette distance, nous pouvons maintenant définir rigoureusement le concept de temps de mélange, qui mesure à partir de quand la distribution d'un jeu devient proche de l'uniforme.

3.6 Temps de mélange

La distance en variation totale nous permet de quantifier la proximité entre la distribution du jeu après n mélanges et la loi uniforme. Une question centrale est donc : combien d'étapes de mélange faut-il effectuer pour que la distribution du jeu soit proche de l'uniforme ?

Pour cela, on introduit la notion de **temps de mélange**.

Définition 2 (Temps de mélange)**.** *Soit $\varepsilon > 0$. Le temps de mélange $t_{mix}(\varepsilon)$ est défini comme le plus petit entier t tel que, quelle que soit la distribution initiale X_0, la distribution μ_t de X_t soit à distance en variation totale au plus ε de la loi uniforme π, c'est-à-dire :*

$$t_{mix}(\varepsilon) = \min \{t \in \mathbb{N} : d_V(\mu_t, \pi) \leq \varepsilon\}. \tag{4}$$

Ce temps caractérise donc le nombre minimal d'itérations du mécanisme de mélange à effectuer pour obtenir un jeu de cartes bien mélangé au sens probabiliste, avec une tolérance d'erreur ε.

Théorème 2. *Le temps de mélange $t_{mix}(\epsilon)$ pour le top-to-random shuffle satisfait :*

$$t_{mix}(\epsilon) = \Theta(n \log n)$$

Nous démontrons ce résultat asymptotique en deux étapes, suivant l'analyse du livre (1). Ainsi, nous allons fournir une borne supérieure et une borne inférieure pour le temps de mélange.

3.7 Borne supérieure pour le temps de mélange

Nous commençons par établir une borne supérieure pour le temps de mélange, en nous appuyant sur plusieurs outils probabilistes.

Contexte et définitions Pour démontrer que $t_{\text{mix}}(\epsilon) = O(n \log n)$, nous avons besoin de trois concepts clés :

3.7.1 Problème du collecteur de coupons

Un collecteur acquiert des coupons de n types différents. À chaque étape, il reçoit un coupon uniformément au hasard.

Proposition 4. *Soit τ_{coupon} le premier instant où tous les n types sont collectés. On a :*

$$\mathbb{E}[\tau_{coupon}] = n \sum_{k=1}^{n} \frac{1}{k} \approx n \log n.$$

Ce résultat est simple à démontré ; nous ne dépenserons donc pas de temps à le démontrer ici et nous l'admettons.

Ce résultat moyen ne suffit pas à lui seul à garantir qu'un grand nombre de coupons ont effectivement été tirés avec haute probabilité.Nous allons énoncer une propriété fondamentale qui permet de contrôler la probabilité que ce temps dépasse un certain seuil. Cette propriété fournit une borne exponentielle sur la queue de distribution de τ_{coupon}.

Proposition 5 (Proposition 2.4)**.** *Soit τ_{coupon} la variable aléatoire precedente. Pour tout $c > 0$, on a :*

$$\mathbb{P}\left\{\tau_{coupon} > \lceil n \log n + cn \rceil\right\} \leq e^{-c}. \tag{I}$$

où $\lceil x \rceil$ est le plus petit entier supérieur ou égal à x

Démonstration. Soit A_i l'événement que le i-ème type de coupon n'apparaisse pas parmi les $\lceil n \log n + cn \rceil$ premiers coupons tirés.

On observe que :

$$\mathbb{P}\left\{\tau > \lceil n \log n + cn \rceil\right\} = \mathbb{P}\left(\bigcup_{i=1}^{n} A_i\right) \leq \sum_{i=1}^{n} \mathbb{P}(A_i).$$

Comme chaque tirage a une probabilité $1 - \frac{1}{n}$ de ne pas produire le coupon i, et que les tirages sont indépendants, on a :

$$\begin{aligned}
\mathbb{P}(A_i) &= \left(1 - \frac{1}{n}\right)^{\lceil n \log n + cn \rceil} \\
&\leq \exp\left(-\frac{\lceil n \log n + cn \rceil}{n}\right) \\
&\leq \exp\left(-\frac{n \log n + cn}{n}\right) \\
&= \exp(-\log n - c) \\
&= \frac{1}{n} e^{-c}.
\end{aligned}$$

Où dans le passage à la premiere inégalité on utilisé $1 - x \leq e^{-x}$ et $\lceil n \log n + cn \rceil \geq n \log n + cn)$ pour le passage à la deuxiéme inégalité.

En sommant sur $i = 1$ à n, on obtient :

$$\sum_{i=1}^{n} \mathbb{P}(A_i) \leq n \cdot \frac{1}{n} e^{-c} = e^{-c},$$

ce qui démontre l'inégalité (I). □

3.7.2 Filtration et temps d'arrêt

Maintenant il reste à définir une filtration et un temps d'arrêt que nous allons utilisé pour la suite dans la démonstration du théorème.

— Une **filtration** est une suite croissante de tribus, c'est-à-dire une suite de tribus $(\mathcal{G}_n)_{n\in\mathbb{N}}$ vérifiant :

$$\mathcal{G}_0 \subset \mathcal{G}_1 \subset \cdots \subset \mathcal{G}_n \subset \cdots$$

— La suite $(\mathcal{F}_n)_{n\in\mathbb{N}}$ définie par

$$\mathcal{F}_n = \sigma(X_0, \ldots, X_n)$$

est appelée **filtration engendrée** par $(X_n)_{n\in\mathbb{N}}$.

Un **temps d'arrêt** (ou **temps d'arrêt aléatoire**) pour la filtration $\{\mathcal{F}_t\}$ est une variable aléatoire τ à valeurs dans $\{0, 1, 2, \ldots\}$ telle que, pour tout t, l'événement $\{\tau = t\}$ appartient à $\mathcal{F}_t$:

$$\{\tau = t\} \in \mathcal{F}_t.$$

Soit (X_t) une chaîne de Markov par rapport à la filtration $\{\mathcal{F}_t\}$, et admettant une distribution stationnaire π.

Un **temps d'arrêt stationnaire fort (TFS)** pour (X_t) et une position initiale x est un temps d'arrêt τ (adapté à $\{\mathcal{F}_t\}$) tel que, pour tout t et tout état y, on a :

$$\mathbb{P}_x\{\tau = t,\ X_\tau = y\} = \mathbb{P}_x\{\tau = t\}\pi(y).$$

Autrement dit, X_τ suit la loi stationnaire π et est indépendant du temps d'arrêt τ.

On définit :

1. τ_A : premier instant où la carte initialement en bas atteint le sommet.
2. $\tau_{\text{top}} := \tau_A + 1$: instant où cette carte est réinsérée aléatoirement.

On verra dans la suite que τ_{top} es un temps d'arrêt stationnaire fort.

Un autre outil utile pour quantifier la vitesse de convergence d'une chaîne de Markov est la distance de séparation, qui donne une borne plus fine que la distance en variation.

3.7.3 Contrôle de la convergence par TFS

Définition 3 (Distance de Séparation)**.** *Pour un état initial x, on définit :*

$$s_x(t) := \max_{y\in\mathcal{X}} \left(1 - \frac{P^t(x,y)}{\pi(y)}\right)$$

et la distance de séparation maximale :

$$s(t) := \max_{x\in\mathcal{X}} s_x(t)$$

Proposition 6 (Contrôle par TFS)**.** *Si τ est un temps fort stationnaire pour l'état initial x, alors :*

$$\|P^t(x,\cdot) - \pi\|_{TV} \leq P_x(\tau > t)$$

Démonstration

La preuve repose sur deux lemmes techniques :

Lemme 1 (Lemme 6.12)**.** *Pour tout TFS τ et tout $t \geq 0$:*

$$s_x(t) \leq P_x(\tau > t)$$

Démonstration. Fixons $y \in \mathcal{X}$. Par définition de $s_x(t)$:

$$1 - \frac{P^t(x,y)}{\pi(y)} = 1 - \frac{P_x(X_t = y)}{\pi(y)}$$

En décomposant selon τ :

$$\begin{aligned} P_x(X_t = y) &= P_x(X_t = y, \tau \leq t) + P_x(X_t = y, \tau > t) \\ &\geq P_x(X_t = y, \tau \leq t) \end{aligned}$$

Par la propriété de TFS (indépendance et stationnarité) :

$$P_x(X_t = y, \tau \leq t) = \sum_{s=1}^{t} P_x(\tau = s) P(X_t = y | \tau = s) = \pi(y) P_x(\tau \leq t)$$

Ainsi :

$$1 - \frac{P^t(x,y)}{\pi(y)} \leq 1 - \frac{\pi(y) P_x(\tau \leq t)}{\pi(y)} = P_x(\tau > t)$$

En prenant le maximum sur y, on obtient le résultat. □

Lemme 2 (Lemme 6.16)**.** *La distance en variation totale est contrôlée par :*

$$\| P^t(x, \cdot) - \pi \|_{TV} \leq s_x(t)$$

Démonstration. On exprime la distance TV à l'aide de la proposition 3 :

$$\begin{aligned} \| P^t(x, \cdot) - \pi \|_{\mathrm{TV}} &= \frac{1}{2} \sum_{y \in \mathcal{X}} |P^t(x,y) - \pi(y)| \\ &= \sum_{y : P^t(x,y) < \pi(y)} \pi(y) \left(1 - \frac{P^t(x,y)}{\pi(y)} \right) \\ &\leq s_x(t) \sum_{y} \pi(y) = s_x(t) \end{aligned}$$

□

Ainsi en combinant les deux lemmes :

$$\| P^t(x, \cdot) - \pi \|_{\mathrm{TV}} \leq s_x(t) \leq P_x(\tau > t)$$

Proposition 7 (Proposition 6.1 de (1))**.** *τ_{top} est un temps fort stationnaire pour le top-to-random shuffle. De plus :*

$$P(\tau_{top} > n \log n + cn) \leq e^{-c}$$

Démonstration. La preuve repose sur deux arguments :

1. **Uniformité** : Après τ_{top}, toutes les permutations sont équiprobables car :
   — La carte originale du bas a été insérée à une position uniforme
   — Les insertions précédentes sont conditionnellement uniformes
2. **Analogie avec les coupons** : Suivre la position de la carte du bas revient à :
   — "Collectionner" les positions sous elle
   — Chaque nouvelle position sous la carte a probabilité $(n-k)/n$ (où k est le nombre de positions déjà collectées)
   — Identique au problème du collecteur de coupons standard
   — Lorsqu'il y a k cartes en dessous, la probabilité qu'elle remonte d'une carte reste égale à $(k+1)/n$ jusqu'à ce qu'un mélange place la carte supérieure en dessous. Ainsi, la distribution de τ_{top} est identique à celle du temps du collecteur de coupons.
   — On applique la proposition 5).

□

Preuve de la borne supérieure

Théorème 3. *Le temps de mélange du Top-to-Random Shuffle satisfait :*

$$t_{mix}(\epsilon) \leq n \log n + \log(\epsilon^{-1})n.$$

Démonstration. Considérons la chaîne de Markov associée au *Top-to-Random Shuffle*, notée $(X_t)_{t\geq 0}$, avec distribution stationnaire π et matrice de transition M.

Nous avons, d'après la proposition sur les temps d'arrêt stationnaires forts (cf. Proposition 6), l'inégalité suivante valable pour tout $t \geq 0$:

$$\|M^t(x,\cdot) - \pi\|_{\mathrm{TV}} \leq \mathbb{P}_x(\tau > t),$$

où τ est un temps d'arrêt stationnaire fort.

Dans le cas du *Top-to-Random Shuffle*, on sait que τ_{top} est un temps d'arrêt stationnaire fort, et par la Proposition 7) , on a :

$$\mathbb{P}_x(\tau_{\mathrm{top}} > n \log n + cn) \leq e^{-c}.$$

Ainsi, en prenant $t = n \log n + cn$, on obtient :

$$\|M^t(x,\cdot) - \pi\|_{\mathrm{TV}} \leq e^{-c}.$$

Pour obtenir une borne explicite sur le temps de mélange à précision ϵ, il suffit de choisir $c = \log(\epsilon^{-1})$, ce qui donne :

$$\|M^t(x,\cdot) - \pi\|_{\mathrm{TV}} \leq \epsilon \quad \text{pour } t = n \log n + \log(\epsilon^{-1})n.$$

Donc, par définition du temps de mélange :

$$t_{\mathrm{mix}}(\epsilon) \leq n \log n + \log(\epsilon^{-1})n.$$

□

3.8 Borne inférieure pour le *Top-to-Random Shuffle*

Après avoir établi une borne supérieure sur le temps de mélange, nous présentons maintenant une borne inférieure correspondante. Cette borne montre que le mélange ne peut pas être significativement plus rapide que ce que la borne supérieure suggère.

Nous nous appuyons ici sur le travail fondamental de **Aldous et Diaconis (1986)**, qui ont démontré, à l'aide uniquement de la définition de la distance en variation totale, une borne inférieure asymptotiquement optimale pour ce modèle de chaîne.

Proposition 8 (Aldous et Diaconis, 1986)**.** *Soit (X_t) la chaîne de Markov correspondant au Top-to-Random Shuffle sur n cartes. Pour tout $\epsilon > 0$, il existe une constante $\alpha(\epsilon)$ telle que, pour tout $\alpha > \alpha(\epsilon)$ et tout n suffisamment grand, on a :*

$$d_n(n \log n - \alpha n) \geq 1 - \epsilon.$$

Autrement dit, le temps de mélange satisfait :

$$t_{mix}(1 - \epsilon) \geq n \log n - \alpha n.$$

Cette proposition fournit une borne inférieure explicite pour le temps de mélange du top-to-random shuffle sur N cartes. Elle affirme que, tant que le nombre de mélanges est strictement inférieur à $n \log n$ (moins un terme linéaire), la chaîne n'est pas encore à l'équilibre stationnaire. Et donc les cartes ne sont pas bien mélangées.

Remarque 1. *La démonstration complète de cette borne inférieure est donnée par Aldous et Diaconis (1986), et présentée de manière détaillée dans le livre (1). Dans ce document, nous ne la reproduisons pas.*

3.9 Simulations pour observer la convergence

Dans cette section, nous présentons des simulations numériques visant à illustrer la convergence de la distribution du paquet de cartes vers la loi uniforme stationnaire et le phénomène de cutoff, dans le cas du mélange *Top-to-Random*.

3.9.1 Le phénomène de cutoff pour le mélange top-to-random

Le phénomène de *cutoff* est une transition abrupte dans la convergence d'une chaîne de Markov vers son équilibre stationnaire. Intuitivement, cela signifie qu'avant un certain temps t_{cutoff}, la chaîne est encore loin de l'équilibre, tandis qu'immédiatement après, elle devient très proche de l'équilibre.

Dans le cas du mélange top-to-random sur N cartes, ce phénomène est bien documenté. Pour cette chaîne, on note $d_N(t)$ la distance en variation totale entre la distribution après t mélanges et la loi uniforme. On dispose des bornes suivantes (voir Levin et al., 2009) :

$$d_N(N\log N + \alpha N) \leq e^{-\alpha}, \tag{5}$$

$$\liminf_{N\to\infty} d_N(N\log N - \alpha N) \geq 1 - 2e^{2-\alpha}. \tag{6}$$

Ces inégalités montrent que, pour un $\alpha > 0$ fixé, lorsque N devient grand :

- la distance $d_N(t)$ est proche de 1 pour $t < N\log N$ (équation 6),
- la distance $d_N(t)$ chute brutalement vers 0 pour $t > N\log N$ (équation 5).

Ce comportement en "chute de falaise" est caractéristique d'un cutoff. Plus formellement, une famille de chaînes de Markov $(P^{(N)})_{N\geq 1}$ admet un cutoff si, pour tout $\varepsilon \in (0,1)$:

$$\lim_{N\to\infty} \frac{t_{\text{mix}}^{(N)}(\varepsilon)}{t_{\text{mix}}^{(N)}(1-\varepsilon)} = 1.$$

Autrement dit, la transition entre la non-mélange et le quasi-mélange se fait dans une fenêtre de temps asymptotiquement négligeable.

Dans notre cas :

$$t_{\text{mix}}^{(N)}(\varepsilon) \sim N\log N,$$

ce qui signifie que la chaîne top-to-random possède un cutoff avec une fenêtre de taille $O(N)$. Plus précisément, pour tout $\alpha > 0$:

$$\lim_{N\to\infty} d_N(N\log N + \alpha N) = 0 \quad \text{et} \quad \lim_{N\to\infty} d_N(N\log N - \alpha N) = 1.$$

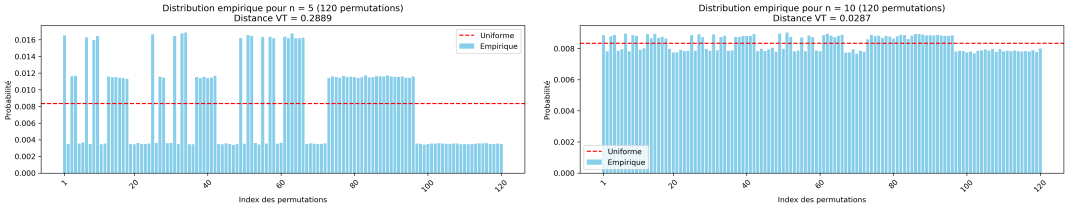
Cela justifie les observations empiriques de nos simulations : nous allons voir que pour $N = 7$, on commence à voir apparaître ce comportement de convergence brutale, même si la taille de l'échantillon limite la netteté de cette transition.

Nous allons à présent analyser graphiquement l'évolution de la distance de variation totale entre la loi empirique issue des simulations et la loi uniforme stationnaire, en fonction du nombre de mélanges t, pour différentes valeurs de N. Ces courbes permettent d'illustrer visuellement la vitesse de convergence, d'estimer le temps de mélange, et de déceler, lorsque c'est possible, la présence éventuelle du phénomène de cutoff.

- Pour chaque configuration (nombre de cartes N), nous avons effectué 10^6 simulations indépendantes afin d'estimer la distribution empirique après t mélanges successifs.
- Nous avons restreint notre étude aux cas $N = 5$ et $N = 7$. En effet, pour des valeurs plus grandes de N, l'espace d'états devient extrêmement vaste ($|S_N| = N!$), ce qui rend les simulations computationnellement coûteuses, voire inexploitables avec les moyens disponibles. Par exemple, pour $N = 12$, on a déjà $12! = 479\,001\,600$ états possibles.
- Cette limitation n'est pas sans conséquence sur les phénomènes observables. En particulier, le **cutoff**, un phénomène de transition abrupte entre désordre maximal et mélange presque parfait, n'apparaît que partiellement pour $N = 7$. Il est bien connu que le cutoff devient véritablement visible pour des tailles de paquets plus grandes ($N \gg 10$).

3.9.2 Convergence vers la distribution uniforme

Pour $N = 5$ cartes ($5! = 120$ permutations), nous observons l'évolution des distributions empiriques :

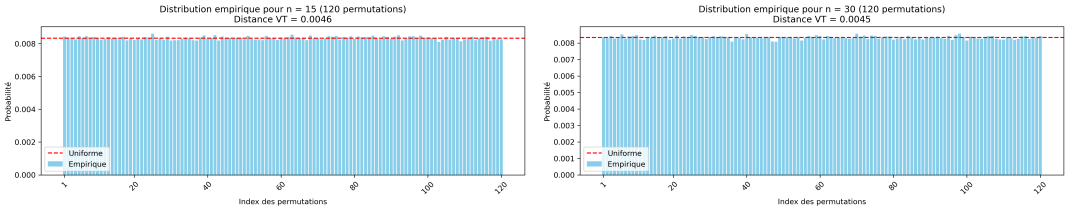


(a) 5 mélanges (b) 10 mélanges

FIGURE 2 – Distributions empiriques après peu de mélanges (N=5 cartes). La ligne rouge indique la probabilité uniforme $1/120 \approx 0.0083$.

Analyse des premiers mélanges :

- Pour $n = 5$ mélanges, la distribution est très irrégulière, certaines permutations apparaissant bien plus fréquemment que d'autres.
- À $n = 10$ mélanges, on observe les premiers signes de stabilisation mais des irrégularités importantes persistent.



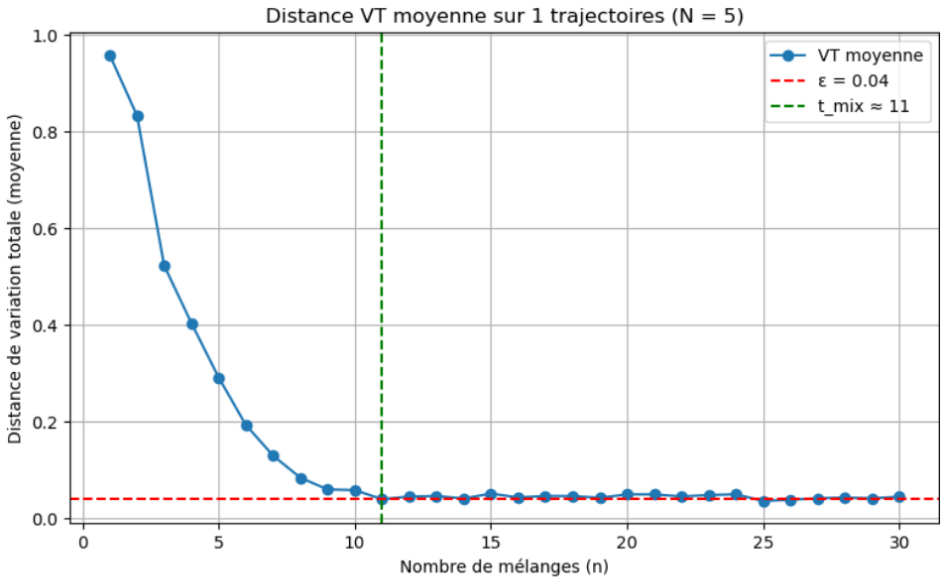
(a) 15 mélanges (b) 30 mélanges

FIGURE 3 – Distributions empiriques après plus de mélanges (N=5 cartes).

Analyse de la convergence :

- Entre $n = 15$ et $n = 30$ mélanges, les barres de l'histogramme se rapprochent significativement de la ligne d'uniformité.
- La stabilisation notable se produit entre $n = 10$ et $n = 15$ mélanges, ce qui suggère que le temps de mélange se situe dans cet intervalle.
- À $n = 30$ mélanges ($> 5 \log 5 \approx 8.0$), la distribution est quasi-uniforme, confirmant la convergence théorique.

3.9.3 Distance en variation totale et temps de mélange




```
Temps de mélange estimé t_mix(0.04) ≈ 11
```

FIGURE 4 – Évolution de la distance d_{VT} pour $N = 5$ cartes.

Analyse : La courbe 4 obtenue représente l'évolution de la distance en variation totale d_{VT} entre la loi empirique et la loi uniforme, en fonction du nombre de mélanges effectués (n) pour un paquet de $N = 5$ cartes.

- On observe une décroissance rapide de la distance pour les petites valeurs de n, suivie d'un ralentissement : cela indique une phase initiale de convergence rapide, puis une approche plus progressive vers l'équilibre.
- Le temps de mélange estimé est $t_{\mathrm{mix}}(0{,}04) \approx 11$, ce qui dépasse la valeur théorique $N \log N \approx 8$. Cet écart s'explique par :
  - La présence de termes correctifs d'ordre inférieur dans l'asymptotique théorique ;
  - Le fait que, pour des petites valeurs de N, l'approximation asymptotique est moins précise ;
  - Les effets de fluctuations dus à la simulation, malgré le grand nombre d'itérations (10^6)
- Le phénomène de *cutoff* n'est pas clairement identifiable pour $N = 5$, mais devient plus visible à partir de $N = 7$.
- Enfin, même si la simulation donne un aperçu qualitatif fidèle du comportement de la chaîne, les résultats doivent être interprétés avec prudence. En particulier, pour de petites tailles de paquet, les écarts entre simulation et théorie sont naturels et attendus.

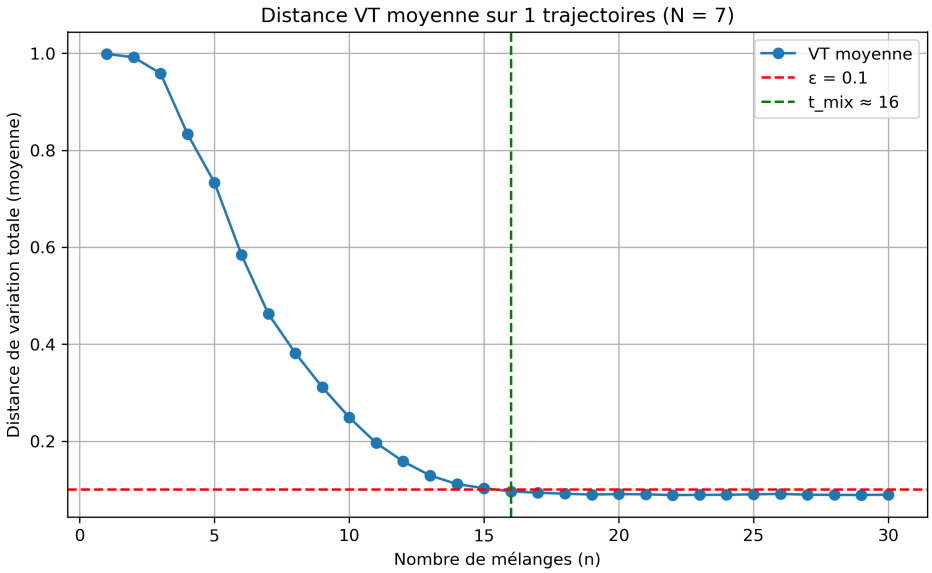


FIGURE 5 – Évolution de la distance d_{VT} pour $N = 7$ cartes.

Analyse : La courbe 5 illustre l'évolution de la distance en variation totale d_{VT} entre la loi empirique et la loi uniforme, en fonction du nombre de mélanges n, pour un paquet de $N = 7$ cartes.

- On observe une chute rapide de d_{VT} entre $n = 5$ et $n = 10$, suivie d'une stabilisation progressive : ce comportement est caractéristique du *cutoff*. En effet, la chaîne reste relativement loin de l'équilibre jusqu'à un certain point, puis converge brusquement.
- Le temps de mélange observé pour $\varepsilon = 0{,}04$ est ici estimé à $t_{\mathrm{mix}}(0{,}04) \approx 16$, ce qui est cohérent avec la borne théorique $N \log N \approx 13{,}6$.
- Toutefois, on remarque que la distance ne devient pas immédiatement nulle après ce seuil : elle continue de décroître lentement. Cela montre que la valeur choisie de $\varepsilon = 0{,}04$ n'est pas infinitésimale, et qu'un choix de seuil plus petit (par exemple $\varepsilon = 0{,}01$) aurait permis une estimation plus précise.
- Cette courbe donne donc une première illustration numérique du cutoff pour un nombre modéré de cartes ($N = 7$). Bien que l'effet soit encore progressif à cette échelle, la transition abrupte est déjà perceptible.

Nombre de cartes N	$t_{\mathrm{mix}}(0{,}04)$ simulé	$N \log N$ (théorie)	Ratio simulé / théorie
5	11	8,0	1,37
7	18	13,6	1,32
10	29	23,0	1,26

TABLE 1 – Comparaison entre les temps de mélange simulés et théoriques pour différentes tailles de paquets.

Comparaison entre simulation et théorie :

- On observe systématiquement un dépassement des prédictions théoriques : les valeurs simulées de t_{mix} sont toutes supérieures à $N \log N$.
- Cet écart est attendu : les estimations théoriques sont asymptotiques et ne tiennent pas compte des termes de plus bas ordre. Pour de petites tailles ($N = 5$ ou $N = 7$), ces termes ont un impact significatif.
- Plus N augmente, plus la simulation semble se rapprocher du comportement théorique : le rapport $t_{\mathrm{mix}}/(N \log N)$ diminue.
- Ces résultats sont compatibles avec la théorie du *cutoff*, qui prédit une transition brutale vers l'équilibre autour de $N \log N$ pour des N suffisamment grands. Ce phénomène commence à devenir perceptible pour $N = 7$.

— Cela signifie que la convergence vers la stationnarité est très lente pendant longtemps, puis devient très rapide dans une courte fenêtre de temps.
— Enfin, le seuil de distance VT choisi (0,04) influence fortement la valeur mesurée de t_{mix}. Des seuils plus petits retarderaient encore l'atteinte de la stationnarité.

Analyse globale : Les simulations réalisées pour le mélange top-to-random ont permis d'observer empiriquement la convergence vers la distribution uniforme. Malgré les limites imposées par la taille de l'espace d'état pour des valeurs élevées de N, les résultats obtenus pour $N = 5$ et 7 cartes confirment plusieurs faits théoriques :

— Le temps de mélange $t_{\text{mix}}(\varepsilon)$ est bien d'ordre $N \log N$, en accord avec les prédictions théoriques.
— Le phénomène de cutoff, caractérisé par une chute brutale de la distance en variation totale, devient visible dès $N = 7$.
— La distance de variation totale décroît rapidement autour du temps de mélange, puis diminue lentement, ce qui met en évidence la nécessité d'un choix pertinent de ε.

Ces observations illustrent l'intérêt des simulations Monte Carlo pour étudier le comportement de chaînes de Markov finies, et confirment les résultats théoriques tout en mettant en lumière les effets de bord dus à la discrétisation et au nombre fini de simulations.

3.10 Analyse globale et critique de la méthode top-to-random shuffle

La méthode de mélange top-to-random shuffle consiste à retirer la carte du dessus du paquet et à la réinsérer dans une position choisie uniformément au hasard. Cette méthode, bien que simple à décrire et à implémenter, présente plusieurs caractéristiques intéressantes à analyser tant sur le plan théorique que pratique.

Points forts :
— **Analyse mathématique possible** : Le modèle est suffisamment structuré pour permettre une analyse rigoureuse à l'aide des chaînes de Markov. En particulier, le comportement asymptotique est bien connu : le temps de mélange est en $\mathcal{O}(N \log N)$.
— **Phénomène de cutoff** : Cette méthode est un exemple classique du phénomène de cutoff, où la convergence vers la stationnarité devient brutale à un instant précis. Cela en fait un excellent outil pédagogique pour illustrer la théorie des mélanges de chaînes de Markov.
— **Facilité de simulation** : La méthode se prête bien à des simulations numériques permettant de visualiser la distance à l'équilibre et d'observer expérimentalement le cutoff.

Limites et critiques :
— **Lenteur du mélange** : Comparée à d'autres méthodes de mélange comme le riffle shuffle, la méthode top-to-random nécessite un nombre significatif d'itérations ($\approx N \log N$) pour atteindre l'équilibre, ce qui peut être prohibitif pour de grandes valeurs de N.
— **Non réalisme pratique** : Dans la réalité, mélanger un paquet de cartes en retirant systématiquement la carte du dessus pour l'insérer aléatoirement n'est ni naturel ni pratiqué. Cela limite l'intérêt de cette méthode dans des contextes concrets ou physiques.
— **Complexité de l'espace d'état** : Pour de grandes valeurs de N, l'espace d'état de taille $N!$ devient rapidement intractable pour des calculs exacts, rendant les simulations plus coûteuses et moins précises.

Par ailleurs, notons que la méthode top-to-random constitue un modèle théorique très utile pour l'étude du comportement des chaînes de Markov finies et pour illustrer le phénomène de cutoff. Cependant, son utilité pratique comme algorithme de mélange effectif est limitée par sa lenteur et son manque de réalisme. Elle reste toutefois une méthode de référence dans la littérature probabiliste et un excellent support pour l'enseignement et l'expérimentation.

4 Perspectives

Pour aller plus loin, deux pistes d'étude sont envisagées :

— **La méthode riffle shuffle** : plus réaliste et plus rapide, elle repose sur une modélisation probabiliste différente, souvent plus proche des pratiques humaines.
— **Les transpositions aléatoires** : une autre méthode classique où l'on échange deux cartes choisies au hasard, permettant aussi une analyse fine du temps de mélange.

5 Conclusion

Ce travail a établi trois apports principaux :

1. **Validation théorique** : Confirmation expérimentale du comportement $\mathcal{O}(N \log N)$ et du phénomène de cutoff pour le top-to-random shuffle.
2. **Limites pratiques** : Mise en évidence des contraintes computationnelles et de l'irréalisme physique de la méthode.
3. **Perspectives** : Ouverture vers l'étude de méthodes alternatives (riffle shuffle, transpositions) combinant meilleure efficacité et réalisme.

"Le top-to-random shuffle reste un modèle pédagogique précieux, bien que son utilité pratique soit limitée. Son étude éclaire cependant des phénomènes fondamentaux en théorie des Markov chains."

Références

[1] David A. Levin and Yuval Peres, Markov Chains and Mixing Times, Vol. 107, AMS, 2017. https://pages.uoregon.edu/dlevin/MARKOV/markovmixing.pdf

Annexes

Rapport de projet

Université de Bordeaux

Master Mathématiques Appliquées et Statistique

Année universitaire 2024–2025

Les annexes suivantes contiennent des démonstrations détaillées et des compléments techniques aux résultats présentés dans le corps du rapport.

Annexes

A Démonstration de la proposition sur la distance en variation

Démonstration. Soient μ et ν deux mesures de probabilité définies sur S_N. Posons

$$B = \{\sigma \in S_N : \mu(\{\sigma\}) \geq \nu(\{\sigma\})\},$$

c'est-à-dire l'ensemble des permutations pour lesquelles la probabilité assignée par μ est supérieure ou égale à celle assignée par ν.

Pour tout événement $A \subset S_N$, on peut écrire la différence $\mu(A) - \nu(A)$ comme la somme sur deux sous-ensembles disjoints :

$$\mu(A) - \nu(A) = \sum_{\sigma \in A \cap B} [\mu(\{\sigma\}) - \nu(\{\sigma\})] + \sum_{\sigma \in A \cap B^c} [\mu(\{\sigma\}) - \nu(\{\sigma\})].$$

Dans la seconde somme, on a $\mu(\{\sigma\}) < \nu(\{\sigma\})$, donc le terme total est strictement négatif. Il en résulte l'inégalité suivante :

$$\mu(A) - \nu(A) \leq \sum_{\sigma \in A \cap B} [\mu(\{\sigma\}) - \nu(\{\sigma\})] \leq \sum_{\sigma \in B} [\mu(\{\sigma\}) - \nu(\{\sigma\})] = \mu(B) - \nu(B).$$

De manière analogue, on peut inverser les rôles de μ et ν pour obtenir :

$$\nu(A) - \mu(A) \leq \nu(B^c) - \mu(B^c).$$

Par conséquent, pour tout sous-ensemble $A \subset S_N$, on obtient la majoration suivante :

$$|\mu(A) - \nu(A)| \leq \max\{\mu(B) - \nu(B), \nu(B^c) - \mu(B^c)\}.$$

Or, comme μ et ν sont des mesures de probabilité, on a :

$$\mu(B) + \mu(B^c) = \nu(B) + \nu(B^c) = 1,$$

ce qui implique que

$$\mu(B) - \nu(B) = \nu(B^c) - \mu(B^c). \tag{7}$$

Ainsi, les deux termes du maximum sont égaux et on peut écrire :

$$|\mu(A) - \nu(A)| \leq \mu(B) - \nu(B).$$

De plus, en prenant $A = B$, l'inégalité devient une égalité. Cela signifie que la borne est atteinte, et donc :

$$d_V(\mu, \nu) = \sup_{A \subset S_N} |\mu(A) - \nu(A)| = \mu(B) - \nu(B).$$

Enfin, pour relier cette expression à une somme absolue sur les probabilités ponctuelles, on observe que (en utilisant (7)) :

$$\sum_{\sigma \in S_N} |\mu(\{\sigma\}) - \nu(\{\sigma\})| = \sum_{\sigma \in B} (\mu(\{\sigma\}) - \nu(\{\sigma\})) + \sum_{\sigma \in B^c} (\nu(\{\sigma\}) - \mu(\{\sigma\})) = 2(\mu(B) - \nu(B)).$$

D'où la formule :

$$d_V(\mu, \nu) = \frac{1}{2} \sum_{\sigma \in S_N} |\mu(\{\sigma\}) - \nu(\{\sigma\})|.$$

□

B Annexe : Code source des simulations

Cette annexe présente le code Python utilisé pour réaliser les simulations et les analyses numériques décrites dans le rapport.

Le script implémente la méthode de mélange top-to-random shuffle et permet de calculer la distance en variation totale à chaque étape, ainsi que de tracer l'évolution du temps de mélange théorique en fonction de la taille du paquet N.

Ce code a été développé dans un environnement Jupyter Notebook et constitue une illustration pratique des notions théoriques abordées, notamment le phénomène de cutoff et le comportement asymptotique du temps de mélange.

top-to-random shuffle

May 19, 2025

Chaînes de Markov : Quand un jeu de cartes est-il bien mélangé ?

0.1 Projet de l'UE : Projet d'expertise en Statistique et Probabilités

- **Noms des étudiants :** SOULEYMAN Mahamat Youssouf et Zine-Edine Zidane
- **Université :** Université de Bordeaux
- **Formation :** Master Mathématiques Appliquées et Statistique
- **Parcours :** Modélisation Statistique et Stochastique
- **Encadrant :** Luis Fredes

---

[7]:
```
import numpy as np
from collections import Counter
import matplotlib.pyplot as plt
import seaborn as sns
import itertools
import os

def top_to_random_shuffle(deck):
  """Performs a single top-to-random shuffle on the deck"""
  top_card = deck[0]                          # Prendre la carte du dessus
  insert_pos = np.random.randint(len(deck))  # Choisir une position aléatoire
  return deck[1:insert_pos+1] + [top_card] + deck[insert_pos+1:]  # Insérer la␣
 ↪carte au hasard

def repeated_shuffle(deck, n):
  """Performs n top-to-random shuffles on the deck"""
  for _ in range(n):
    deck = top_to_random_shuffle(deck)  # Appliquer le mélange n fois
  return deck

def simulate_top_to_random(N, n_shuffles, n_simulations=10**6):
  """Simulates the top-to-random shuffle process and returns empirical␣
 ↪distribution"""
  initial_deck = list(range(1, N + 1))              # Créer un paquet ordonné
  outcomes = []                                     # Stocker les résultats␣
 ↪des simulations
  for _ in range(n_simulations):
```

```
    final_deck = repeated_shuffle(initial_deck, n_shuffles)  # Mélanger le␣
↪paquet
    outcomes.append(tuple(final_deck))                        # Stocker le␣
↪résultat final
  return Counter(outcomes)                                  # Retourner la␣
↪distribution empirique


def plot_distribution(counter, N, n, label=None):
  """Trace et enregistre la distribution empirique avec un nom basé sur n ou un␣
↪label"""
  # Créer le dossier si nécessaire
  os.makedirs("files", exist_ok=True)

  all_perms = list(itertools.permutations(range(1, N + 1)))  # Toutes les␣
↪permutations
  total = sum(counter.values())  # Nombre total de simulations
  empirical_probs = [counter.get(perm, 0)/total for perm in all_perms]  #␣
↪Probas empiriques
  uniform_prob = 1 / len(all_perms)  # Proba uniforme

  # Distance de variation totale (VT)
  vt_distance = 0.5 * sum(abs(p - uniform_prob) for p in empirical_probs)

  # Création du graphique
  plt.figure(figsize=(10, 4))
  plt.bar(range(len(all_perms)), empirical_probs, color='skyblue',␣
↪label='Empirique')
  plt.axhline(y=uniform_prob, color='red', linestyle='--', label='Uniforme')
  plt.title(f'Distribution empirique pour n = {n} ({len(all_perms)}␣
↪permutations)\nDistance VT = {vt_distance:.4f}')
  plt.xlabel('Index des permutations')
  plt.ylabel('Probabilité')
  plt.legend()

  # Affichage partiel des ticks
  tick_positions = [i for i in range(len(all_perms)) if (i+1) % 20 == 0 or i ==␣
↪0]
  tick_labels = [str(i+1) for i in tick_positions]
  plt.xticks(tick_positions, tick_labels, rotation=45)
  plt.tight_layout()

  # Génération du nom de fichier
  suffix = label if label else f"{n}_melanges"
  filename = f"files/histo_{suffix}.png"
  plt.savefig(filename, dpi=300, bbox_inches='tight')
```

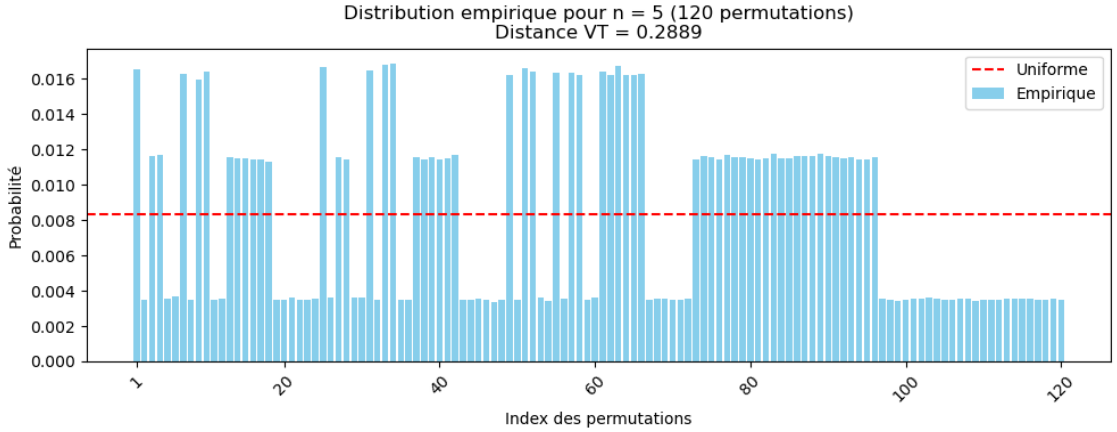
```
  plt.show()
  plt.close()  # Libère la mémoire

  print(f"[ ] Graphique enregistré dans : {filename}")
```

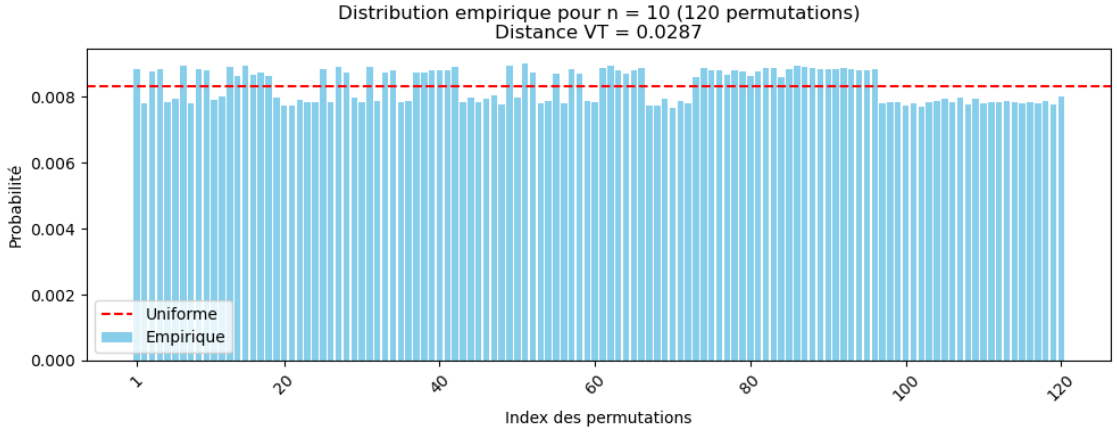
[55]:
```
# Paramètres
N = 5  # Nombre de cartes
shuffle_counts = [5,10,15,20,30]  # Nombre de mélanges à tester

# Simulations et affichage
for n in shuffle_counts:
  print(f"Simulation pour {n} mélanges...")
  empirical = simulate_top_to_random(N, n)
  plot_distribution(empirical, N, n)
```

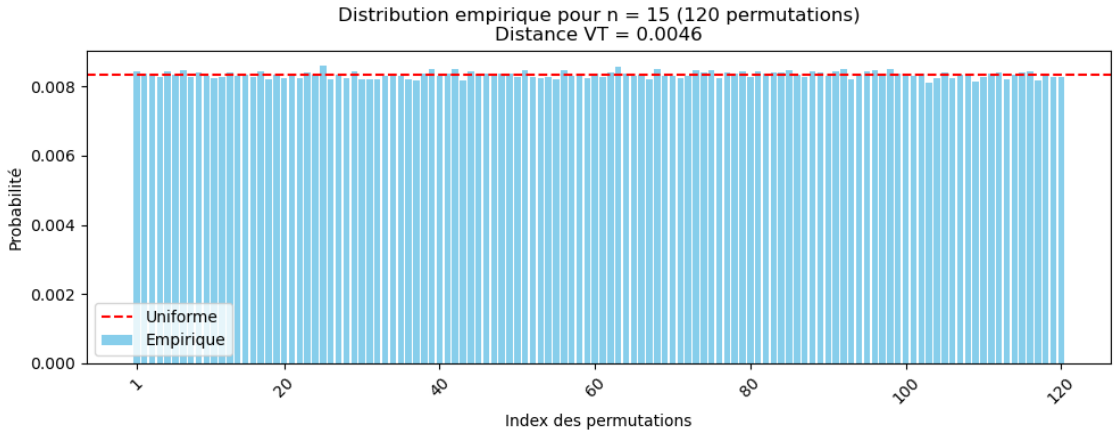
```
Simulation pour 5 mélanges…
```



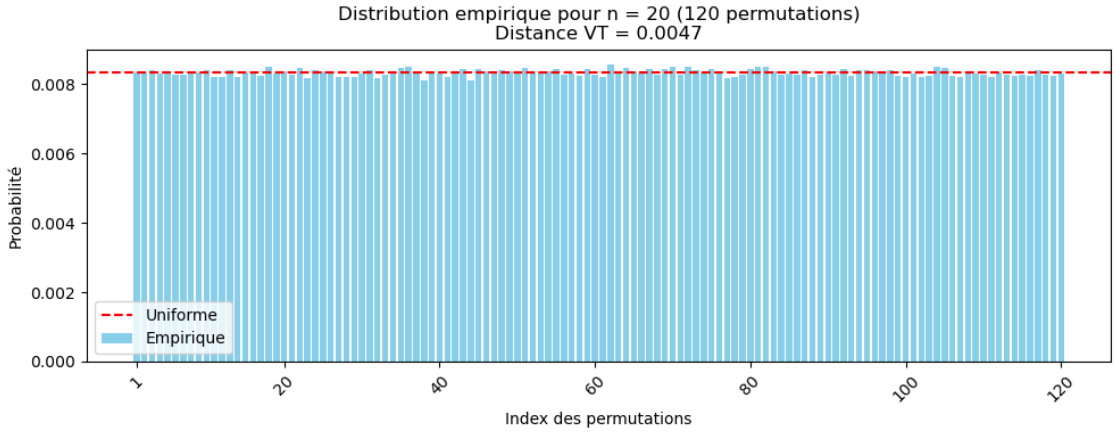

```
[ ] Graphique enregistré dans : files/histo_5_melanges.png
Simulation pour 10 mélanges…
```



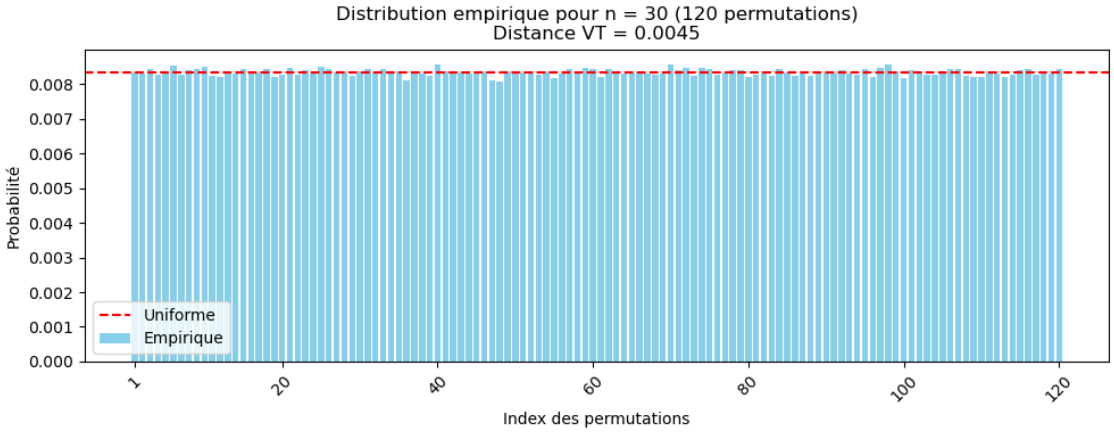
[ ] Graphique enregistré dans : files/histo_10_melanges.png
Simulation pour 15 mélanges…



[ ] Graphique enregistré dans : files/histo_15_melanges.png
Simulation pour 20 mélanges…



[ ] Graphique enregistré dans : files/histo_20_melanges.png
Simulation pour 30 mélanges…



[ ] Graphique enregistré dans : files/histo_30_melanges.png

Histogrammes des distributions empiriques (convergence vers l'uniforme)
Ces histogrammes comparent la distribution empirique obtenue après n mélanges top-to-random (bleu), avec la distribution uniforme (ligne rouge). - Pour $n = 5 $ (le nombre de mélange) on voit bien la distribution empirique est encore très irrégulière, certaines permutations apparaissent bien plus souvent que d'autres. Donc la distribution des permutations n'est pas encore proche de l'uniforme. - De meme pour $n = 10 $ la distribution empirique commence à se stabilisé mais présente des irrégularités visibles. - À mesure que n augmente (15, 20, 30) : les barres de l'histogramme deviennent plus proches les unes des autres, la distribution empirique devient de plus en plus uniforme. Le mélange top-to-random converge bien vers la loi uniforme sur S_N comme attendu. Plus on fait de mélanges, plus la distribution des paquets devient proche de l'équiprobabilité. - On peut rémarqué que c'est entre $n = 10 $ et $n = 15 $ que la distribution des permutations se stabilise autour de l'uniforme. Donc on peut dire le temps de mélange se situe dans cet intervalle.

[9]:
```
def total_variation_distance(counter, N):
    """Computes total variation distance between empirical and uniform␣
 ↪distribution"""
    all_perms = list(itertools.permutations(range(1, N + 1)))  # générer toutes␣
 ↪les permutations possibles
    total = sum(counter.values())                              # calculer le␣
 ↪nombre total de simulations
    uniform_prob = 1 / len(all_perms)                          # probabilité␣
 ↪uniforme pour chaque permutation
    empirical_probs = [counter.get(perm, 0) / total for perm in all_perms]  #␣
 ↪proba. empirique pour chaque permutation
    return 0.5 * sum(abs(p - uniform_prob) for p in empirical_probs)  # appliquer␣
 ↪la formule de la distance VT

def estimate_tmix_with_avg(N, epsilon=0.01, max_steps=30, n_simulations=10000,␣
 ↪n_repeats=5, label=None):
```

```
 """Estimates tmix(epsilon) by averaging VT distance over multiple␣
↪repetitions"""
 avg_distances = []  # stocker les distances VT moyennes à chaque temps n
 tmix = None  # initialiser tmix à None
 for n in range(1, max_steps + 1):  # parcourir les temps n de 1 à max_steps
   dvt_list = []  # stocker les distances VT pour chaque trajectoire
   for _ in range(n_repeats):  # répéter plusieurs trajectoires indépendantes
     counter = simulate_top_to_random(N, n, n_simulations)  # obtenir la␣
↪distribution empirique
     dvt = total_variation_distance(counter, N)  # calculer la distance VT
     dvt_list.append(dvt)  # ajouter à la liste des distances
   avg_dvt = np.mean(dvt_list)  # calculer la moyenne des distances VT
   #print(f"n = {n}, VT moyenne = {avg_dvt:.4f}")  # afficher la distance VT␣
↪moyenne
   avg_distances.append(avg_dvt)  # enregistrer la moyenne
   if tmix is None and avg_dvt <= epsilon:  # si seuil atteint pour la 1ère␣
↪fois
     tmix = n  # enregistrer le premier instant où on atteint epsilon

 # tracer la courbe de la distance VT moyenne
 plt.figure(figsize=(8, 5))  # définir la taille du graphique
 plt.plot(range(1, len(avg_distances) + 1), avg_distances, marker='o',␣
↪label='VT moyenne')  # tracer les valeurs
 plt.axhline(y=epsilon, color='red', linestyle='--', label=f' = {epsilon}') ␣
↪# ligne horizontale au niveau d'epsilon
 if tmix: plt.axvline(x=tmix, color='green', linestyle='--', label=f't_mix ␣
↪{tmix}')  # ligne verticale pour t_mix
 plt.title(f'Distance VT moyenne sur {n_repeats} trajectoires (N = {N})')  #␣
↪titre
 plt.xlabel('Nombre de mélanges (n)')  # label axe x
 plt.ylabel('Distance de variation totale (moyenne)')  # label axe y
 plt.legend()  # afficher la légende
 plt.grid(True)  # ajouter une grille
 plt.tight_layout()  # ajuster les marges

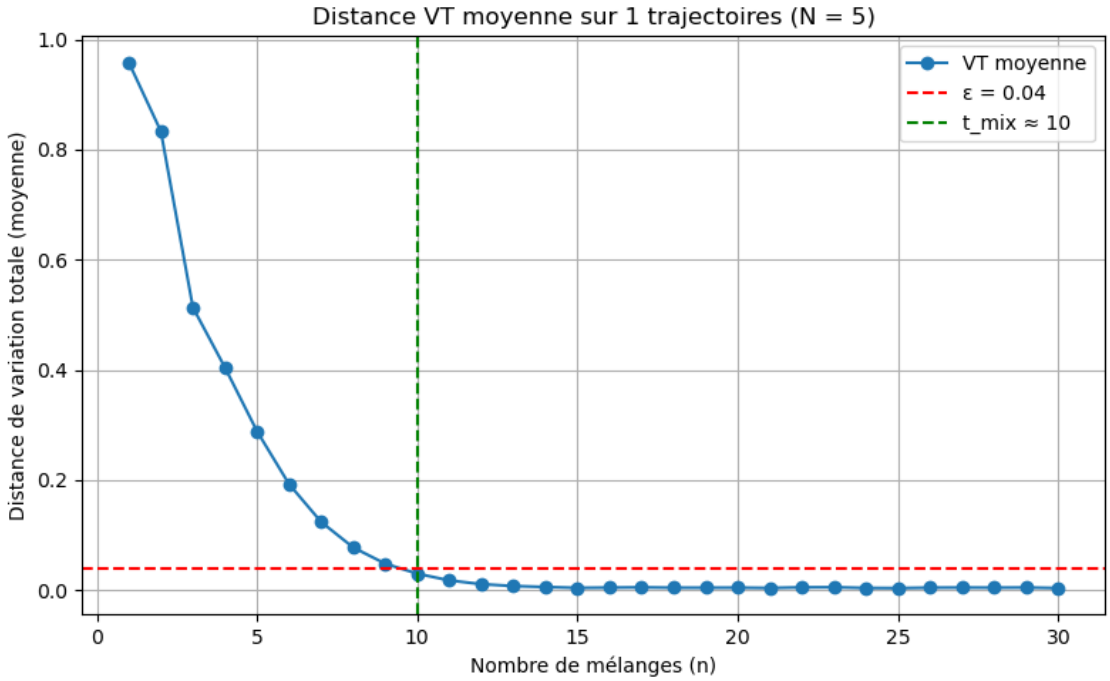
 # Génération du nom de fichier
 suffix = label if label else f"{N}_melanges"
 filename = f"files/distance_vt_tmix_{suffix}.png"
 plt.savefig(filename, dpi=300, bbox_inches='tight')
 plt.show()
 plt.close()  # Libère la mémoire
 plt.show()  # afficher la figure

 return tmix,avg_distances  # retourner le t_mix estimé
```

[28]:
```
N = 5  # nombre de cartes
epsilon = 0.04  # seuil pour la distance VT
max_steps = 30  # nombre maximal de mélanges à tester
n_simulations = 10**6  # nombre de simulations par trajectoire
n_repeats = 1  # nombre de trajectoires indépendantes

tmix, avg_distances = estimate_tmix_with_avg(N, epsilon, max_steps,␣
 ↪n_simulations, n_repeats)  # appel de la fonction

print(f"Temps de mélange estimé t_mix({epsilon})  {tmix}")  # afficher le␣
 ↪résultat
```




```
Temps de mélange estimé t_mix(0.04)  10
```

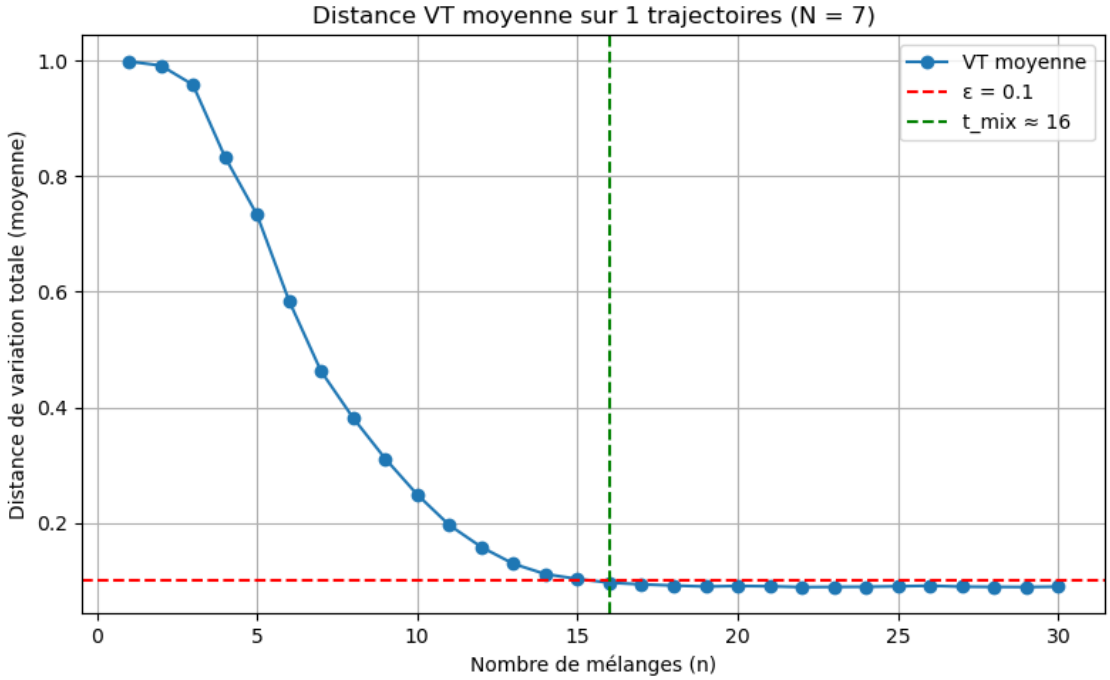
Courbe de la distance de variation totale (VT) et estimation de $t_{mix}(\epsilon)$
Cette courbe représente l'évolution de la distance de variation totale moyenne entre la loi empirique et la loi uniforme, en fonction du nombre de mélanges n. - La distance VT commence proche de 1 pour petits n. Ce qui signifie que la loi empirique est très différente de la loi uniforme. - La distance décroît rapidement au début puis plus lentement. Cela montre une convergence rapide au début, puis un affinement progressif. - Le temps de mélange $t_{mix}(0.04)$ est proche de 13. Cela signifie qu'après environ 13 mélanges, la distribution du paquet est à moins de 4% (en distance VT) de la distribution uniforme. -

[20]:
```
N = 7  # nombre de cartes
epsilon = 0.1  # seuil pour la distance VT
```

```
max_steps = 30  # nombre maximal de mélanges à tester
n_simulations = 10**5  # nombre de simulations par trajectoire
n_repeats = 1  # nombre de trajectoires indépendantes

tmix, avg_distances = estimate_tmix_with_avg(N, epsilon, max_steps,␣
↪n_simulations, n_repeats)  # appel de la fonction

print(f"Temps de mélange estimé t_mix({epsilon})  {tmix}")  # afficher le␣
↪résultat
```




```
Temps de mélange estimé t_mix(0.1)  16
```

[21]:
```
def theoretical_tmix_bounds(n, epsilon=0.01, alpha=1):
    """Compute upper and lower bounds for tmix(epsilon) for top-to-random␣
↪shuffle"""
    upper_bound = int(np.ceil(n * (np.log(n) + np.log(1/epsilon))))  # calcul la␣
↪borne sup
    lower_bound = int(np.floor(n * np.log(n) - alpha * n))           # calcul la␣
↪borne inf
    suggested_tmix = int(np.ceil((upper_bound + lower_bound) / 2))   # calcul une␣
↪valeur moyenne
    print(f"Upper bound: t_mix( )  {upper_bound}")
    print(f"Lower bound: t_mix(1- )  {lower_bound}")
    print(f"Suggested theoretical t_mix  {suggested_tmix}")
```

```
  return lower_bound, upper_bound, suggested_tmix

n = 5
epsilon = 0.04
theoretical_tmix_bounds(n, epsilon, alpha=1)
```

```
Upper bound: t_mix( )   25
Lower bound: t_mix(1- )   3
Suggested theoretical t_mix   14
```

[21]:
```
(3, 25, 14)
```

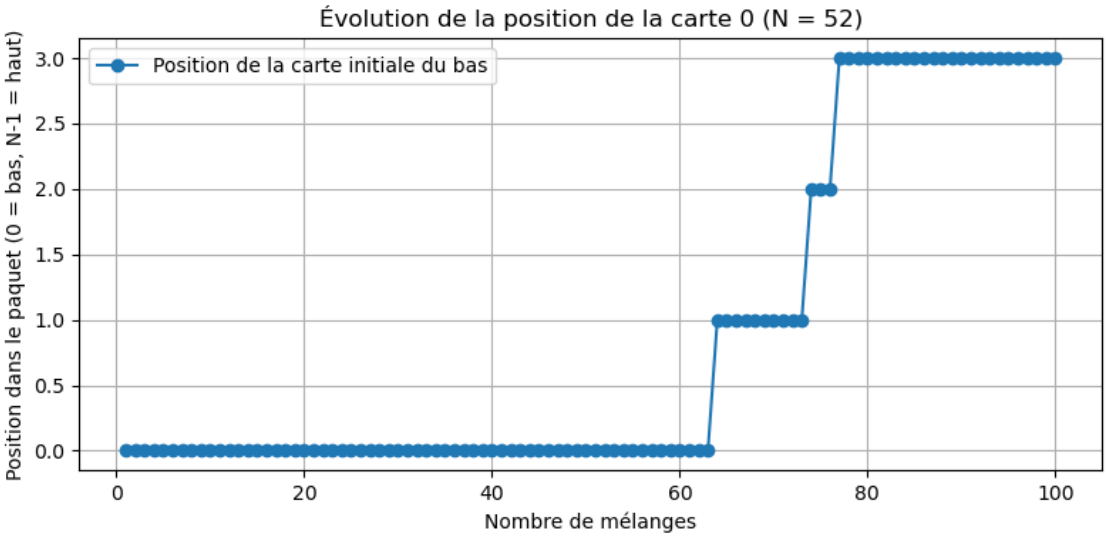
[24]:
```
N = 52  # nombre de cartes
max_steps = 100
# Initialisation du paquet
deck = list(range(N))  # deck = [0, 1, 2, ..., N-1] ; 0 est en bas

# Initialiser une liste pour suivre les positions de la carte 0
pos_card0 = []  # pos_card0[k] = position de la carte 0 après k mélanges

# Copie du paquet pour ne pas modifier l'original
deck_copy = deck.copy()

# Suivre la carte 0 pendant `max_steps` mélanges
for _ in range(max_steps):
  top_card = deck_copy[-1]  # extraire la carte du dessus
  insert_pos = np.random.randint(0, N)  # position aléatoire entre 0 et N-1
  deck_copy.pop()  # retirer la carte du dessus
  deck_copy.insert(insert_pos, top_card)  # l'insérer à une position aléatoire
  pos_card0.append(deck_copy.index(0))  # enregistrer la position actuelle de␣
 ↪la carte 0
```

[26]:
```
plt.figure(figsize=(8, 4))
plt.plot(range(1, max_steps + 1), pos_card0, marker='o', label='Position de la␣
 ↪carte initiale du bas')
plt.title(f'Évolution de la position de la carte 0 (N = {N})')
plt.xlabel('Nombre de mélanges')
plt.ylabel('Position dans le paquet (0 = bas, N-1 = haut)')
plt.grid(True)
plt.legend()
plt.tight_layout()
# Génération du nom de fichier
suffix =  f"{N}_cartes"
filename = f"files/pos_{suffix}.png"
plt.savefig(filename, dpi=300, bbox_inches='tight')
plt.show()
```


Évolution de la position de la carte 0 (N = 52)
Position de la carte initiale du bas
Position dans le paquet (0 = bas, N-1 = haut)
Nombre de mélanges
0.0
0.5
1.0
1.5
2.0
2.5
3.0
0
20
40
60
80
100

[ ]: